

CLOUD COMPUTING

LAB NO # 12



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

LAB NO # 12

Task 0: Lab Setup (Codespace & GH CLI)

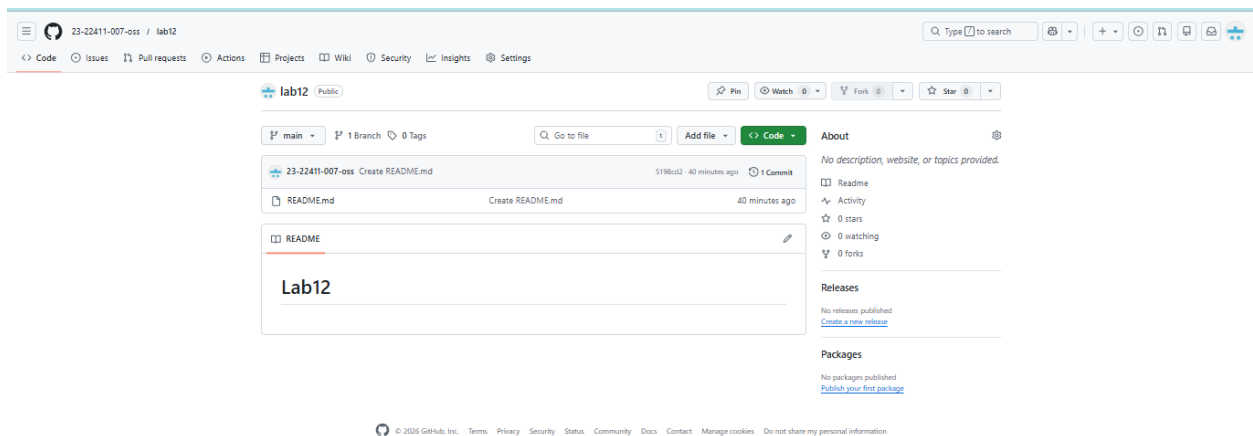
Step 1 — Open GitHub Codespaces Environment

What you need to do

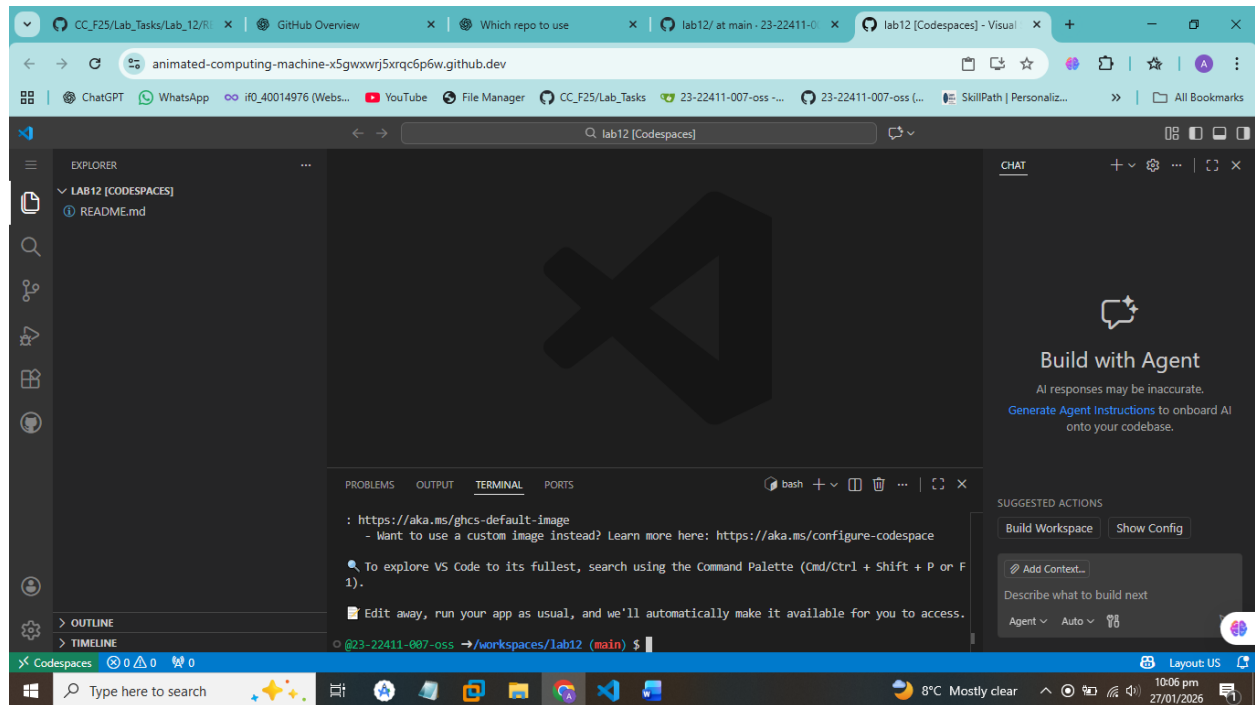
You must first access a **Codespace environment**, because all GH CLI commands must run **inside it**.

How to do it (Click steps)

1. Open **GitHub.com**
2. Click your **profile picture (top-right)**
3. Click **Your repositories**
4. Click **New** (create a temporary repo if needed)



1. Open:
👉 <https://github.com/codespaces>
2. Click **New codespace**



Step 2 — Verify GH CLI Is Installed

Ensures the **GitHub CLI (gh)** is available inside Codespace.

Type this command in the Codespace terminal:

`gh --version`

```
● @23-22411-007-oss → /workspaces/lab12 (main) $ gh --version
gh version 2.83.1 (2025-11-13)
https://github.com/cli/cli/releases/tag/v2.83.1
○ @23-22411-007-oss → /workspaces/lab12 (main) $
```

Step 3 — Authenticate GH CLI (Inside Codespace ONLY)

`gh auth login`

Follow these exact selections:

1. **What account do you want to log into?**
→ GitHub.com
2. **Preferred protocol for Git operations?**
→ HTTPS
3. **Authenticate GitHub CLI how?**
→ Login with a web browser

You'll see a **one-time code**.

What to click

1. Copy the code shown
2. Click the URL shown in terminal
3. Paste the code
4. Click **Authorize GitHub**

```
@23-22411-007-oss →/workspaces/lab12 (main) $ gh auth login
? Where do you use GitHub? GitHub.com
The value of the GITHUB_TOKEN environment variable is being used for authentication.
To have GitHub CLI store credentials instead, first clear the value from the environment.
• @23-22411-007-oss →/workspaces/lab12 (main) $ gh auth status
github.com
✓ Logged in to github.com account 23-22411-007-oss (GITHUB_TOKEN)
- Active account: true
- Git operations protocol: https
- Token: ghu_*****
@23-22411-007-oss →/workspaces/lab12 (main) $
```

Screenshot REQUIRED

 Save as:

task0_codespace_create_and_list.png

Step 4 — List All Codespaces

What this step does

Shows available Codespaces and their names.

Type:

gh codespace list

```
• Token: ghu_
• @23-22411-007-oss →/workspaces/lab12 (main) $ gh codespace list
NAME              DISPLAY NAME      REPOSITORY        BRANCH  STATE      CREATED AT
animated-computin...  animated comput...  23-22411-007-os...  main    Available  about 1 hour ago
@23-22411-007-oss →/workspaces/lab12 (main) $
```

Screenshot REQUIRED

 Save as: **task0_codespace_create_and_list.png**

Step 5 — SSH into the Codespace

Connects you **inside the Codespace shell**, where all future lab tasks **MUST** be done.

Type:

gh codespace ssh -c <codespace_name>

📌 Example:

gh codespace ssh -c happy-bassoon

```
@23-22411-007-oss →/workspaces/lab12 (main) $ gh codespace ssh
? Choose codespace: 23-22411-007-oss/lab12 [main]: animated computing-machine
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
@23-22411-007-oss →/workspaces/lab12 (main) $ whoami
uname -a
pwd
codespace
Linux codespaces-9af19d 6.8.0-1030-azure #35~22.04.1-Ubuntu SMP Mon May 26 18:08:30 UTC 2025 x86_
64 x86_64 x86_64 GNU/Linux
/workspaces/lab12
@23-22411-007-oss →/workspaces/lab12 (main) $
```

📷 Screenshot REQUIRED

📁 Save as:

task0_codespace_ssh_connected.png

Task 1 — Organize Terraform code into separate files

STEP 1: Create Project Directory

Run:

mkdir -p ~/Lab12

cd ~/Lab12

```
@23-22411-007-oss →/workspaces/lab12 (main) $ mkdir -p ~/Lab12
cd ~/Lab12
@23-22411-007-oss →~/Lab12 $ ped
bash: ped: command not found
@23-22411-007-oss →~/Lab12 $ pwd
/home/codespace/Lab12
```

Screenshot 📷

- **Filename:** task1_project_directory.png

STEP 2: Create Required Files

Run:

touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars entry-script.sh

ls -la

```
@23-22411-007-oss →~/Lab12 $ touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars entry-script.sh
@23-22411-007-oss →~/Lab12 $ ls -la
total 12
drwxr-xr-x 2 codespace codespace 4096 Jan 28 16:50 .
drwxr-x--- 1 codespace codespace 4096 Jan 28 16:49 ..
-rw-r--r-- 1 codespace codespace  0 Jan 28 16:50 entry-script.sh
-rw-r--r-- 1 codespace codespace  0 Jan 28 16:50 locals.tf
-rw-r--r-- 1 codespace codespace  0 Jan 28 16:50 main.tf
-rw-r--r-- 1 codespace codespace  0 Jan 28 16:50 outputs.tf
-rw-r--r-- 1 codespace codespace  0 Jan 28 16:50 terraform.tfvars
-rw-r--r-- 1 codespace codespace  0 Jan 28 16:50 variables.tf
@23-22411-007-oss →~/Lab12 $
```

Screenshot 

- **Filename:** task1_files_created.png

STEP 3: Create variables.tf

Edit file:

nano variables.tf

Paste:

variable "vpc_cidr_block" {}

variable "subnet_cidr_block" {}

variable "availability_zone" {}

variable "env_prefix" {}

variable "instance_type" {}

variable "public_key" {}

variable "private_key" {}

Save → CTRL+O → Enter → CTRL+X

```
@23-22411-007-oss →~/Lab12 $ nano variables.tf
@23-22411-007-oss →~/Lab12 $ cat variables.tf
variable "vpc_cidr_block" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
variable "instance_type" {}
variable "public_key" {}
variable "private_key" {}
@23-22411-007-oss →~/Lab12 $
```

Screenshot 

- **Filename:** task1_variables_tf.png

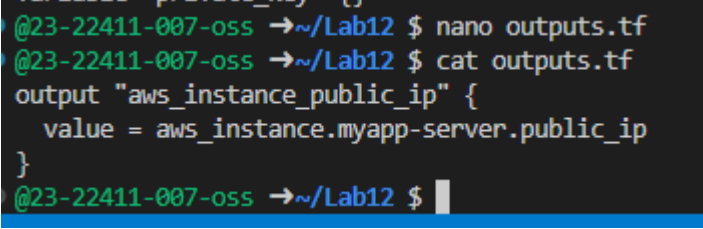
STEP 4: Create outputs.tf

Edit:

nano outputs.tf

Paste:

```
output "aws_instance_public_ip" {  
    value = aws_instance.myapp-server.public_ip  
}
```



A screenshot of a terminal window with a dark background and light-colored text. The prompt is '@23-22411-007-oss →~/Lab12 \$'. The user enters 'nano outputs.tf', followed by 'cat outputs.tf'. The output of the cat command is displayed: 'output "aws_instance_public_ip" {', 'value = aws_instance.myapp-server.public_ip', and '}'. The prompt then changes to '@23-22411-007-oss →~/Lab12 \$' with a cursor at the end.

Screenshot 📷

- **Filename:** task1_outputs_tf.png

STEP 5: Create locals.tf

Edit:

nano locals.tf

Paste:

```
locals {  
    my_ip = "${chomp(data.http.my_ip.response_body)}/32"  
}
```

```
data "http" "my_ip" {  
    url = "https://icanhazip.com"  
}
```

```
@23-22411-007-oss →~/Lab12 $ nano locals.tf
@23-22411-007-oss →~/Lab12 $ cat locals.tf
locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"
}

data "http" "my_ip" {
  url = "https://icanhazip.com"
}
@23-22411-007-oss →~/Lab12 $
```

Screenshot 

- **Filename:** task1_locals_tf.png

STEP 6: Create terraform.tfvars

Edit:

nano terraform.tfvars

Paste:

vpc_cidr_block = "10.0.0.0/16"

subnet_cidr_block = "10.0.10.0/24"

availability_zone = "me-central-1a"

env_prefix = "dev"

instance_type = "t3.micro"

public_key = "~/.ssh/id_ed25519.pub"

private_key = "~/.ssh/id_ed25519"

```
@23-22411-007-oss →~/Lab12 $ cat terraform.tfvars
vpc_cidr_block = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix = "dev"
instance_type = "t3.micro"
public_key = "~/.ssh/id_ed25519.pub"
private_key = "~/.ssh/id_ed25519"
@23-22411-007-oss →~/Lab12 $
```

Screenshot 

- **Filename:** task1_terraform_tfvars.png

STEP 7: Create main.tf

Edit:

nano main.tf

Paste exactly the code your task provides (no changes).

```
provider "aws" {  
    shared_config_files    = ["~/.aws/config"]  
    shared_credentials_files = ["~/.aws/credentials"]  
}  
  
resource "aws_vpc" "myapp_vpc" {  
    cidr_block = var.vpc_cidr_block  
    tags = {  
        Name = "${var.env_prefix}-vpc"  
    }  
}  
  
resource "aws_subnet" "myapp_subnet_1" {  
    vpc_id    = aws_vpc.myapp_vpc.id  
    cidr_block = var.subnet_cidr_block  
    availability_zone = var.availability_zone  
    tags = {  
        Name = "${var.env_prefix}-subnet-1"  
    }  
}  
  
resource "aws_default_route_table" "main_rt" {  
    default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
```

```
route {  
  cidr_block = "0.0.0.0/0"  
  gateway_id = aws_internet_gateway.myapp_igw.id  
}  
tags = {  
  Name = "${var.env_prefix}-rt"  
}  
}
```

```
resource "aws_internet_gateway" "myapp_igw" {  
  vpc_id = aws_vpc.myapp_vpc.id  
  tags = {  
    Name = "${var.env_prefix}-igw"  
  }  
}
```

```
resource "aws_default_security_group" "default_sg" {  
  vpc_id = aws_vpc.myapp_vpc.id
```

```
  ingress {  
    from_port = 22  
    to_port   = 22  
    protocol  = "tcp"  
    cidr_blocks = [local.my_ip]  
  }
```

```
ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}

egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    prefix_list_ids = []
}

tags = {
    Name = "${var.env_prefix}-default-sg"
}
}
```

```
resource "aws_key_pair" "ssh-key" {
    key_name = "serverkey"
    public_key = file(var.public_key)
}
```

```
resource "aws_instance" "myapp-server" {
    ami = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
    instance_type = var.instance_type
}
```

```
subnet_id    = aws_subnet.myapp_subnet_1.id

security_groups = [aws_default_security_group.default_sg. id]

availability_zone = var.availability_zone

associate_public_ip_address = true

key_name = aws_key_pair.ssh-key. key_name

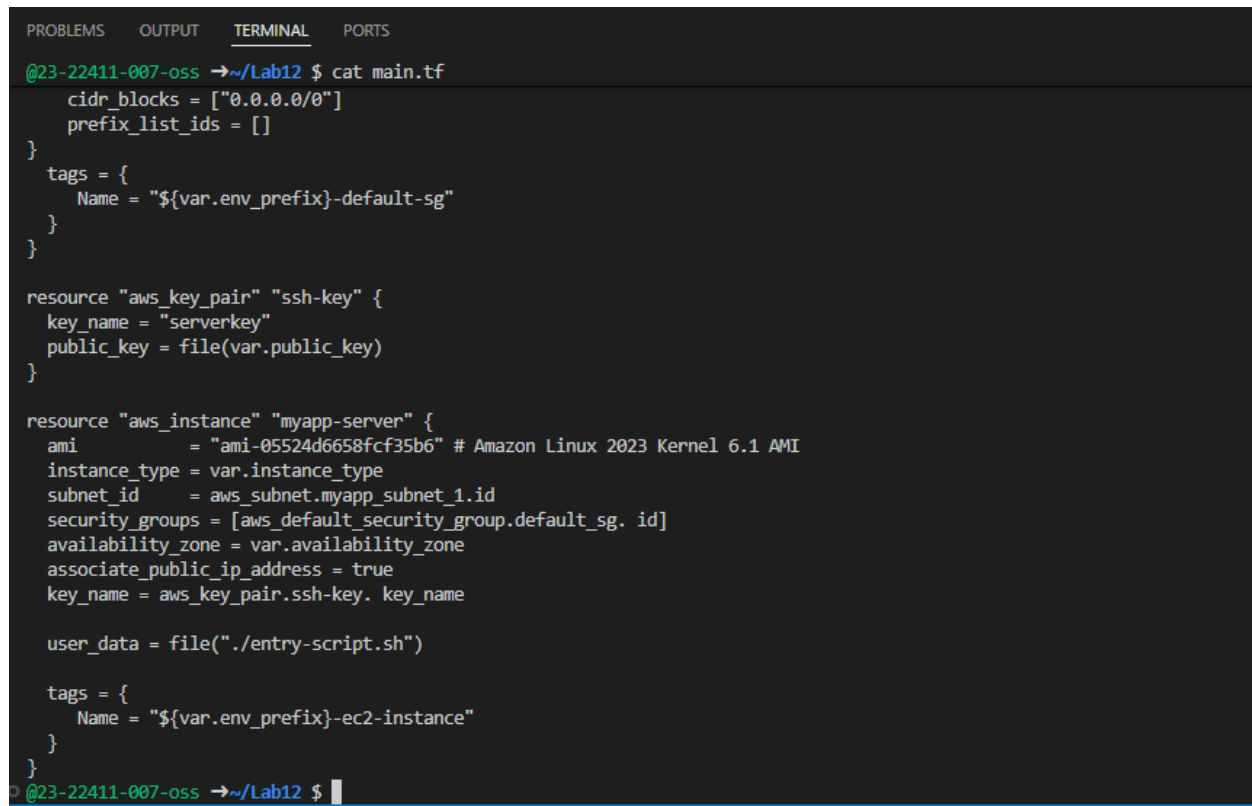

user_data = file("./entry-script.sh")


tags = {

    Name = "${var.env_prefix}-ec2-instance"

}

}
```



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The prompt is '@23-22411-007-oss → ~/Lab12 \$'. The user has entered 'cat main.tf', and the contents of the file are displayed. The code is Terraform configuration for an AWS instance, including variables for CIDR blocks, prefix list IDs, tags, and resource definitions for an AWS key pair and an AWS instance. The instance resource is named 'myapp-server' and includes attributes for AMI, instance type, subnet ID, security groups, availability zone, public IP address association, key name, user data, and tags. The terminal ends with the prompt '@23-22411-007-oss → ~/Lab12 \$'.

```
PROBLEMS  OUTPUT  TERMINAL  PORTS
@23-22411-007-oss → ~/Lab12 $ cat main.tf
cidr_blocks = ["0.0.0.0/0"]
prefix_list_ids = []
}
tags = {
    Name = "${var.env_prefix}-default-sg"
}
}

resource "aws_key_pair" "ssh-key" {
    key_name = "serverkey"
    public_key = file(var.public_key)
}

resource "aws_instance" "myapp-server" {
    ami           = "ami-05524d6658f35b6" # Amazon Linux 2023 Kernel 6.1 AMI
    instance_type = var.instance_type
    subnet_id     = aws_subnet.myapp_subnet_1.id
    security_groups = [aws_default_security_group.default_sg. id]
    availability_zone = var.availability_zone
    associate_public_ip_address = true
    key_name = aws_key_pair.ssh-key. key_name

    user_data = file("./entry-script.sh")

    tags = {
        Name = "${var.env_prefix}-ec2-instance"
    }
}
@23-22411-007-oss → ~/Lab12 $
```

Screenshot 📷

- **Filename:** task1_main_tf.png

STEP 8: Create entry-script.sh

Edit:

nano entry-script.sh

Paste:

```
#!/bin/bash
```

```
set -e
```

```
yum update -y
```

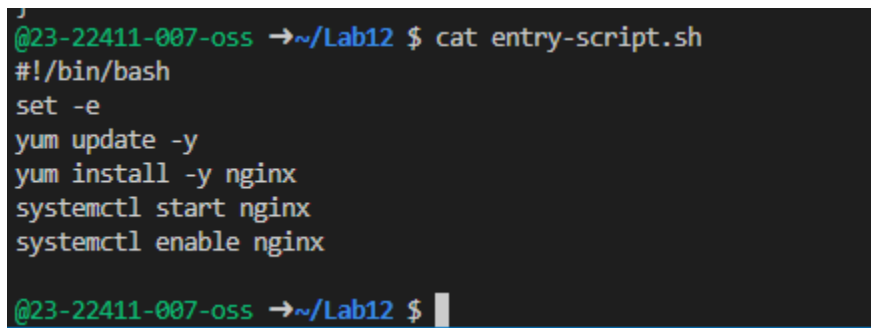
```
yum install -y nginx
```

```
systemctl start nginx
```

```
systemctl enable nginx
```

Make executable:

```
chmod +x entry-script.sh
```

A screenshot of a terminal window with a dark background. The prompt is '@23-22411-007-oss → ~/Lab12 \$'. The user has entered 'cat entry-script.sh' and the output is displayed line by line: '#!/bin/bash', 'set -e', 'yum update -y', 'yum install -y nginx', 'systemctl start nginx', and 'systemctl enable nginx'. The prompt is now '@23-22411-007-oss → ~/Lab12 \$' with a cursor.

Screenshot 📷

- **Filename:** task1_entry_script.png

STEP 9: Generate SSH Key Pair

Run:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
```

```

@23-22411-007-oss →~/Lab12 $ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:u2F0w1PW021W4ke080XIbnMCCL5DwzLg0IhGh8s8BMc codespace@codespaces-9af19d
The key's randomart image is:
+--[ED25519 256]--+
|...                |
|..E                |
|+ .. . . .        |
|*.00 . 0 . 0 = =   |
|0*0 . 0 5 + = B .  |
|. .. = * + B       |
|      @ . = * .    |
|      + * 0 0 +    |
|      o            |
+-----[SHA256]-----+
@23-22411-007-oss →~/Lab12 $

```

Screenshot 🖼️

- **Filename:** task1_ssh_keygen.png

STEP 10: Initialize Terraform

Run:

terraform init

```

@23-22411-007-oss →~/Lab12 $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Finding latest version of hashicorp/http...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@23-22411-007-oss →~/Lab12 $

```

Screenshot

- **Filename:** task1_terraform_init.png

STEP 11: Apply Terraform Configuration

Run:

terraform apply -auto-approve

```
@23-22411-007-oss →~/Lab12 $ terraform init
terraform apply -auto-approve
}

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ aws_instance_public_ip = (known after apply)
aws_key_pair.ssh_key: Creating...
aws_vpc.myapp_vpc: Creating...
aws_key_pair.ssh_key: Creation complete after 1s [id=serverkey]
aws_vpc.myapp_vpc: Creation complete after 2s [id=vpc-038ef02b9dd9cb64f]
aws_internet_gateway.myapp_igw: Creating...
aws_subnet.myapp_subnet_1: Creating...
aws_default_security_group.default_sg: Creating...
aws_internet_gateway.myapp_igw: Creation complete after 1s [id=igw-0c5538a407e6f2aac]
aws_default_route_table.main_rt: Creating...
aws_subnet.myapp_subnet_1: Creation complete after 1s [id=subnet-00c54ebd70056caf1]
aws_default_route_table.main_rt: Creation complete after 2s [id=rtb-03c60daec285e4702]
aws_default_security_group.default_sg: Creation complete after 4s [id=sg-0db0ebde5845ddad6]
aws_instance.myapp_server: Creating...
aws_instance.myapp_server: Still creating... [00m10s elapsed]
aws_instance.myapp_server: Creation complete after 14s [id=i-02f9b2769706b13cc]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "3.28.44.10"
@23-22411-007-oss →~/Lab12 $
```

Screenshot

- **Filename:** task1_terraform_apply.png

STEP 12: Display Terraform Output

Run:

terraform output

```
@23-22411-007-oss →~/Lab12 $ terraform output
aws_instance_public_ip = "3.28.44.10"
@23-22411-007-oss →~/Lab12 $
```

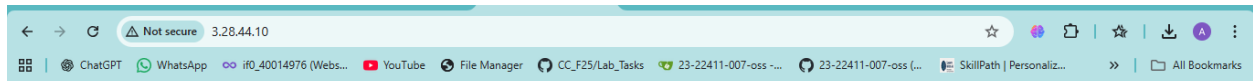
Screenshot

- **Filename:** task1_terraform_output.png

STEP 13: Test Nginx in Browser

Open browser:

http://<PUBLIC-IP>



You should see “Welcome to nginx!”

Screenshot 📷

- **Filename:** task1_nginx_browser.png

STEP 14: Destroy Resources

Run:

terraform destroy

Type:

Yes

```
@23-22411-007-oss →~/Lab12 $ terraform destroy
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_default_route_table.main_rt: Destroying... [id=rtb-03c60daec285e4702]
aws_default_route_table.main_rt: Destruction complete after 0s
aws_instance.myapp_server: Destroying... [id=i-02f9b2769706b13cc]
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0c5538a407e6f2aac]
aws_instance.myapp_server: Still destroying... [id=i-02f9b2769706b13cc, 00m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0c5538a407e6f2aac, 00m10s elapsed]
aws_instance.myapp_server: Still destroying... [id=i-02f9b2769706b13cc, 00m20s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0c5538a407e6f2aac, 00m20s elapsed]
aws_instance.myapp_server: Still destroying... [id=i-02f9b2769706b13cc, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0c5538a407e6f2aac, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Destruction complete after 39s
aws_instance.myapp_server: Still destroying... [id=i-02f9b2769706b13cc, 00m40s elapsed]
aws_instance.myapp_server: Destruction complete after 42s
aws_key_pair.ssh_key: Destroying... [id=serverkey]
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-00c54ebd70056caf1]
aws_default_security_group.default_sg: Destroying... [id=sg-0db0ebde5845ddad6]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh_key: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-038ef02b9dd9cb64f]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
@23-22411-007-oss →~/Lab12 $
```

Screenshot

- **Filename:** task1_terraform_destroy.png
- Must show:
 - Destroy complete!

Task 2 — Use remote-exec provisioner

STEP 1: Modify main.tf

1. Open main.tf:

nano main.tf

2. **Locate the aws_instance "myapp-server" resource.**
It currently has:

```
user_data = file("./entry-script.sh")
```

3. **Replace the user_data line with the remote-exec block:**

```
resource "aws_instance" "myapp-server" {
```

```
ami      = "ami-05524d6658fcf35b6"
instance_type = var.instance_type
subnet_id  = aws_subnet.myapp_subnet_1.id
security_groups = [aws_default_security_group.default_sg.id]
availability_zone = var.availability_zone
associate_public_ip_address = true
key_name = aws_key_pair.ssh-key.key_name
```

```
connection {
  type      = "ssh"
  user      = "ec2-user"
  private_key = file(var.private_key)
  host      = self.public_ip
}
```

```
provisioner "remote-exec" {
  inline = [
    "sudo yum update -y",
    "sudo yum install -y nginx",
    "sudo systemctl start nginx",
    "sudo systemctl enable nginx"
  ]
}
```

```
tags = {
  Name = "${var.env_prefix}-ec2-instance"
}
}
```

4. Save the file → CTRL+O → Enter → CTRL+X

```
@23-22411-007-oss →~/Lab12 $ cat main.tf
subnet_id      = aws_subnet.myapp_subnet_1.id
vpc_security_group_ids = [aws_default_security_group.default_sg.id]
availability_zone = var.availability_zone
associate_public_ip_address = true
key_name       = aws_key_pair.ssh_key.key_name

connection {
  type     = "ssh"
  user     = "ec2-user"
  private_key = file(var.private_key)
  host     = self.public_ip
  timeout  = "5m"
}

provisioner "remote-exec" {
  inline = [
    "sudo yum update -y",
    "sudo yum install -y nginx",
    "sudo systemctl start nginx",
    "sudo systemctl enable nginx"
  ]
}

tags = {
  Name = "${var.env_prefix}-ec2-instance"
}
}
```

Filename: task2_main_tf_remote_exec.png

STEP 2: Apply Terraform Configuration

1. Initialize Terraform (if not already initialized):

terraform init

2. Apply changes:

terraform apply -auto-approve

- Terraform will now:
 - Create/update the EC2 instance
 - SSH into it using your key
 - Run the commands to install & start Nginx

```

@23-22411-007-oss →~/Lab12 $ terraform init
terraform apply -auto-approve
aws_instance.myapp_server (remote-exec): libunwind-1.4.0-5.amzn2023.0.3.x86_64
aws_instance.myapp_server (remote-exec): nginx-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp_server (remote-exec): nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp_server (remote-exec): nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch
aws_instance.myapp_server (remote-exec): nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

aws_instance.myapp_server (remote-exec): Complete!
aws_instance.myapp_server (remote-exec): Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
aws_instance.myapp_server: Still creating... [01m00s elapsed]
aws_instance.myapp_server: Creation complete after 1m0s [id=i-01243f8c0bb659c18]

Warning: Deprecated attribute
  on locals.tf line 7, in locals:
   7:   my_ip = "${chomp(data.http.my_ip.body)}/32"

The attribute "body" is deprecated. Refer to the provider documentation for details.
(and 2 more similar warnings elsewhere)

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "3.28.40.237"
@23-22411-007-oss →~/Lab12 $

```

Filename: task2_terraform_apply.png

STEP 3: Display Terraform Output

terraform output

```

@23-22411-007-oss →~/Lab12 $ terraform output
aws_instance_public_ip = "3.28.40.237"
@23-22411-007-oss →~/Lab12 $

```

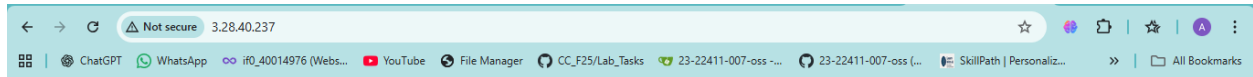
Screenshot Required:

- **Filename:** task2_terraform_output.png

STEP 4: Test Nginx in Browser

- Open browser:

http://<PUBLIC-IP>



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Screenshot Required:

- **Filename:** task2_nginx_browser.png

Task 3 — Use file and local-exec provisioners

STEP 1: Modify main.tf to use all three provisioners

1. Open main.tf:

```
nano main.tf
```

2. Find the aws_instance "myapp-server" resource.
3. Replace the connection + remote-exec block with:

```
resource "aws_instance" "myapp-server" {  
  ami          = "ami-05524d6658fcf35b6"  
  instance_type = var.instance_type  
  subnet_id    = aws_subnet.myapp_subnet_1.id  
  security_groups = [aws_default_security_group.default_sg.id]  
  availability_zone = var.availability_zone  
  associate_public_ip_address = true  
  key_name = aws_key_pair.ssh-key.key_name  
  
  connection {  
    type      = "ssh"  
    user      = "ec2-user"  
    private_key = file(var.private_key)  
    host      = self.public_ip  
  }  
}
```

```
}
```

```
provisioner "file" {  
  source    = "./entry-script.sh"  
  destination = "/home/ec2-user/entry-script-on-ec2.sh"  
}
```

```
provisioner "remote-exec" {  
  inline = [  
    "sudo chmod +x /home/ec2-user/entry-script-on-ec2.sh",  
    "sudo /home/ec2-user/entry-script-on-ec2.sh"  
  ]  
}
```

```
provisioner "local-exec" {  
  command = <<-EOF  
    echo Instance ${self.id} with public IP ${self.public_ip} has been created  
  EOF  
}
```

```
tags = {  
  Name = "${var.env_prefix}-ec2-instance"  
}  
}
```

4. Save → CTRL+O → Enter → CTRL+X

```
@23-22411-007-oss →~/Lab12 $ cat main.tf
host      = self.public_ip
}

# File Provisioner
provisioner "file" {
  source      = "./entry-script.sh"
  destination = "/home/ec2-user/entry-script-on-ec2.sh"
}

# Remote-Exec Provisioner
provisioner "remote-exec" {
  inline = [
    "sudo chmod +x /home/ec2-user/entry-script-on-ec2.sh",
    "sudo /home/ec2-user/entry-script-on-ec2.sh"
  ]
}

# Local-Exec Provisioner
provisioner "local-exec" {
  command = <<-EOF
    echo Instance ${self.id} with public IP ${self.public_ip} has been created
  EOF
}

tags = {
  Name = "${var.env_prefix}-ec2-instance"
}
}
```

Screenshot Required:

- **Filename:** task3_main_tf_all_provisioners.png

STEP 2: Apply the Configuration

terraform apply -auto-approve

- Terraform will:
 1. Upload entry-script.sh with **file** provisioner
 2. Run it on EC2 with **remote-exec**
 3. Log instance info locally with **local-exec**

```
@23-22411-007-oss →~/Lab12 $ terraform init
terraform plan
terraform apply -auto-approve

aws_instance.myapp_server (remote-exec): Complete!
aws_instance.myapp_server (remote-exec): Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
aws_instance.myapp_server: Still creating... [01m00s elapsed]
aws_instance.myapp_server: Provisioning with 'local-exec'...
aws_instance.myapp_server (local-exec): Executing: ["/bin/sh" "-c" "echo Instance i-0b596f8720dfee63c with public IP 51.112.44.204 has been created\n"]
aws_instance.myapp_server (local-exec): Instance i-0b596f8720dfee63c with public IP 51.112.44.204 has been created
aws_instance.myapp_server: Creation complete after 1m0s [id=i-0b596f8720dfee63c]

Warning: Deprecatd attribute
  on locals.tf line 7, in locals:
   7:   my_ip = "${chomp(data.http.my_ip.body)}/32"

The attribute "body" is deprecated. Refer to the provider documentation for details.
(and 2 more similar warnings elsewhere)

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.

Outputs:

aws_instance_public_ip = "51.112.44.204"
@23-22411-007-oss →~/Lab12 $
```

Screenshot Required:

- **Filename:** task3_terraform_apply.png

STEP 3: Display Terraform Output

terraform output

- Copy the **public IP** of your EC2

```
@23-22411-007-oss →~/Lab12 $ terraform output
aws_instance_public_ip = "51.112.44.204"
@23-22411-007-oss →~/Lab12 $
```

Screenshot Required:

- **Filename:** task3_terraform_output.png

STEP 4: Test Nginx in Browser

- Open browser:

http://<PUBLIC-IP>

- You should see **Nginx default page**



Screenshot Required:

- **Filename:** task3_nginx_browser.png

STEP 5: Destroy Resources

terraform destroy

- Type yes when prompted.

```
@23-22411-007-oss →~/Lab12 $ terraform destroy
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_route_table_association.subnet_assoc: Destroying... [id=rtbassoc-01184d0144310114c]
aws_instance.myapp_server: Destroying... [id=i-0b596f8720dfec63c]
aws_route_table_association.subnet_assoc: Destruction complete after 1s
aws_route_table.main_rt: Destroying... [id=rtb-0f1a6206b54d8ad59]
aws_route_table.main_rt: Destruction complete after 1s
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0815f4ad64d860762]
aws_instance.myapp_server: Still destroying... [id=i-0b596f8720dfec63c, 00m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0815f4ad64d860762, 00m10s elapsed]
aws_instance.myapp_server: Still destroying... [id=i-0b596f8720dfec63c, 00m20s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0815f4ad64d860762, 00m20s elapsed]
aws_instance.myapp_server: Still destroying... [id=i-0b596f8720dfec63c, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Destruction complete after 29s
aws_instance.myapp_server: Destruction complete after 32s
aws_key_pair.ssh_key: Destroying... [id=serverkey]
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-0a605b280ac22bdd0]
aws_security_group.default_sg: Destroying... [id=sg-0fe9e0b6cf5a12810]
aws_key_pair.ssh_key: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 1s
aws_security_group.default_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-015928495f707b4c2]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 8 destroyed.
@23-22411-007-oss →~/Lab12 $
```

Screenshot Required:

- **Filename:** task3_terraform_destroy.png

STEP 6: Restore user_data in main.tf

1. Open main.tf:

nano main.tf

2. Replace **connection + provisioners** with:

```
user_data = file("./entry-script.sh")
```

3. Save → CTRL+O → Enter → CTRL+X

```
@23-22411-007-oss →~/Lab12 $ cat main.tf
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.env_prefix}-sg"
  }
}

# Key Pair
resource "aws_key_pair" "ssh_key" {
  key_name     = "serverkey"
  public_key   = file(var.public_key)
}

# EC2 Instance
resource "aws_instance" "myapp_server" {
  ami                  = "ami-05524d6658fcf35b6"
  instance_type        = var.instance_type
  subnet_id            = aws_subnet.myapp_subnet_1.id
  security_groups      = [aws_security_group.default_sg.id]
  availability_zone    = var.availability_zone
  associate_public_ip_address = true
  key_name             = aws_key_pair.ssh_key.key_name

  user_data = file("./entry-script.sh")
}
@23-22411-007-oss →~/Lab12 $
```

Screenshot Required:

- **Filename:** task3_main_tf_restored.png

Task 4 — Create Terraform modules (subnet module)

STEP 1: Create Module Directory Structure

Run:

```
mkdir -p modules/subnet
```

```
touch modules/subnet/main.tf modules/subnet/variables.tf modules/subnet/outputs.tf
```

Check structure:

tree modules

or if tree not installed:

ls -R modules

```
@23-22411-007-oss →~/Lab12 $ mkdir -p modules/subnet
touch modules/subnet/main.tf modules/subnet/variables.tf modules/subnet/outputs.tf
@23-22411-007-oss →~/Lab12 $ tree modules
# or if tree not installed:
ls -R modules
modules
├── subnet
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
└── 2 directories, 3 files

modules:
subnet

modules/subnet:
main.tf outputs.tf variables.tf
@23-22411-007-oss →~/Lab12 $ cat modules/subnet/variables.tf
```

Screenshot 🖼️

- **Filename:** task4_module_structure.png

STEP 2: Create modules/subnet/variables.tf

Edit:

nano modules/subnet/variables.tf

Paste:

variable "vpc_id" {}

variable "subnet_cidr_block" {}

variable "availability_zone" {}

variable "env_prefix" {}

variable "default_route_table_id" {}

Save & Exit

```

@23-22411-007-oss →~/Lab12 $ nano modules/subnet/variables.tf
@23-22411-007-oss →~/Lab12 $ cat modules/subnet/variables.tf
variable "vpc_id" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
variable "default_route_table_id" {}
@23-22411-007-oss →~/Lab12 $

```

Screenshot 📷

- **Filename:** task4_subnet_variables.png

STEP 3: Create modules/subnet/main.tf

Edit:

nano modules/subnet/main.tf

Paste:

```

resource "aws_subnet" "myapp_subnet_1" {
  vpc_id    = var.vpc_id
  cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  map_public_ip_on_launch = true
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = var.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
}

```

```

tags = {
    Name = "${var.env_prefix}-rt"
}

resource "aws_internet_gateway" "myapp_igw" {
    vpc_id = var.vpc_id
    tags = {
        Name = "${var.env_prefix}-igw"
    }
}

```

Save & Exit

```

@23-22411-007-oss →~/Lab12 $ cat modules/subnet/main.tf
resource "aws_subnet" "myapp_subnet_1" {
    vpc_id      = var.vpc_id
    cidr_block  = var.subnet_cidr_block
    availability_zone = var.availability_zone
    map_public_ip_on_launch = true
    tags = {
        Name = "${var.env_prefix}-subnet-1"
    }
}

resource "aws_default_route_table" "main_rt" {
    default_route_table_id = var.default_route_table_id

    route {
        cidr_block = "0.0.0.0/0"
        gateway_id = aws_internet_gateway.myapp_igw.id
    }
    tags = {
        Name = "${var.env_prefix}-rt"
    }
}

resource "aws_internet_gateway" "myapp_igw" {
    vpc_id = var.vpc_id
    tags = {
        Name = "${var.env_prefix}-igw"
    }
}
@23-22411-007-oss →~/Lab12 $

```

Screenshot

- **Filename:** task4_subnet_main.png

STEP 4: Create modules/subnet/outputs.tf

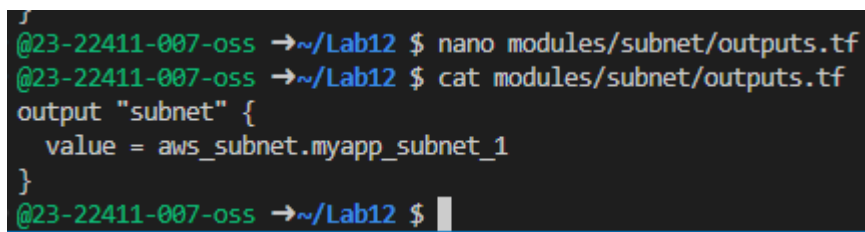
Edit:

nano modules/subnet/outputs.tf

Paste:

```
output "subnet" {  
    value = aws_subnet.myapp_subnet_1  
}
```

Save & Exit



```
@23-22411-007-oss →~/Lab12 $ nano modules/subnet/outputs.tf  
@23-22411-007-oss →~/Lab12 $ cat modules/subnet/outputs.tf  
output "subnet" {  
    value = aws_subnet.myapp_subnet_1  
}  
@23-22411-007-oss →~/Lab12 $
```

Screenshot 📷

- **Filename:** task4_subnet_outputs.png

STEP 5: Modify Root main.tf to Use Module

1. Open root main.tf:

nano main.tf

2. **Remove the old subnet / route table / internet gateway resources.**

3. **Add module block:**

```
module "myapp-subnet" {  
    source = "./modules/subnet"  
  
    vpc_id = aws_vpc.myapp_vpc.id  
    subnet_cidr_block = var.subnet_cidr_block  
    availability_zone = var.availability_zone  
    env_prefix = var.env_prefix  
    default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id  
}
```

4. **Update EC2 resource:**

subnet_id = module.myapp-subnet.subnet.id

```
@23-22411-007-oss →~/Lab12 $ cat main.tf

# -----
# SUBNET MODULE
# -----
module "myapp-subnet" {
  source           = "./modules/subnet"
  vpc_id           = aws_vpc.myapp_vpc.id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix       = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}

# -----
# EC2 INSTANCE
# -----
resource "aws_instance" "myapp_server" {
  ami                = var.ami
  instance_type      = var.instance_type
  subnet_id          = module.myapp-subnet.subnet.id
  security_groups    = [aws_security_group.default_sg.id]
  availability_zone   = var.availability_zone
  associate_public_ip_address = true
  key_name            = aws_key_pair.ssh_key.key_name

  connection {
    type      = "ssh"
    user      = "ec2-user"
    private_key = file(var.private_key)
  }
}
```

Screenshot 

- **Filename:** task4_main_tf_with_module.png

STEP 6: Initialize Terraform (to load module)

terraform init

```
@23-22411-007-oss →~/Lab12 $ terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/http from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@23-22411-007-oss →~/Lab12 $
```

Screenshot

- **Filename:** task4_terraform_init.png

STEP 7: Apply Configuration

terraform apply -auto-approve

```
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-092e9efb0e6e0cfee]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "51.112.49.92"
```

Screenshot

- **Filename:** task4_terraform_apply.png

STEP 8: Display Terraform Output

terraform output

```
aws_instance_public_ip = "51.112.49.92"
```

Screenshot

- **Filename:** task4_terraform_output.png

STEP 9: Test Nginx in Browser

- Open browser:

http://<PUBLIC-IP>



Screenshot

- **Filename:** task4_nginx_browser.png

Task 5: Create webserver module

Create webserver module structure: Tree modules

```

modules
├── subnet
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
└── webserver
    ├── main.tf
    ├── outputs.tf
    └── variables.tf

3 directories, 6 files

```

Add webserver variables

```

lab_12 > modules > webserver > variables.tf > variable "vpc_id"
1  variable "env_prefix" {}
2  variable "instance_type" {}
3  variable "availability_zone" {}
4  variable "public_key" {}

```

Add webserver resources

```

main.tf lab_12 U  main.tf ../webserver U X  outputs.tf U
lab_12 > modules > webserver > main.tf > ...
1  resource "aws_security_group" "web_sg" {
2      vpc_id      = var.vpc_id
3      name        = "${var.env_prefix}-web-sg-${var.instance_suffix}"
4      description = "Security group for web server allowing HTTP, HTTPS and SSH"
5
6      ingress {
7          from_port = 22
8          to_port   = 22
9          protocol  = "tcp"
10         cidr_blocks = [var.my_ip]

```

Add webserver outputs

```

main.tf lab_12 U  main.tf ../webserver U  outputs.tf U X
lab_12 > modules > webserver > outputs.tf > ...
1  output "aws_instance" {
2      value = aws_instance.myapp-server
3  }

```

Use webserver module in root main.tf

```

main.tf lab_12 U X  main.tf ../webserver U  outputs.tf U
lab_12 > main.tf > ...
22  module "myapp-webserver" {
23      source      = "../modules/webserver"
24      env_prefix   = var.env_prefix
25      instance_type = var.instance_type
26      availability_zone = var.availability_zone
27      public_key   = var.public_key
28      my_ip        = local.my_ip
29      vpc_id       = aws_vpc.myapp_vpc.id
30      subnet_id    = module.myapp-subnet.subnet.id
31      script_path  = "../entry-script.sh"

```

Update root outputs

```
main.tf lab_12 U | outputs.tf X | main.tf ../webserver U
lab_12 > outputs.tf > output "webserver_public_ip"
1  output "webserver_public_ip" {
2    value = module.myapp-webserver.aws_instance.public_ip
3  }
4  output "aws_instance_public_ip" {
5    value = aws_instance.myapp-server.public_ip
6  }
```

Initialize Terraform: Terraform init

```
- Using previously-installed hashicorp/aws v6.28.0
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Apply Terraform: Terraform apply -auto-approve

```
aws_instance.myapp-server: Destruction complete after 50s
aws_key_pair.ssh-key: Destroying... [id=serverkey]
aws_default_security_group.default_sg: Destroying... [id=sg-09ac2a296f58e305c]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh-key: Destruction complete after 1s

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

Outputs:

webserver_public_ip = "3.29.67.212"
```

View output: Terraform output

```
webserver_public_ip = "3.29.67.212"
```

Verify nginx



Destroy resources: Terraform destroy

```
module.myapp-webserver.aws_security_group.web_sg: Destroying... [id=sg-00531ad45b7a1ffa9]
module.myapp-webserver.aws_key_pair.ssh-key: Destruction complete after 0s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 0s
module.myapp-webserver.aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-009464bdd9258365b]
aws_vpc.myapp_vpc: Destruction complete after 0s

Destroy complete! Resources: 7 destroyed.
```

Task 6: Configure HTTPS with self-signed certificates

Update entry script with SSL configuration

```
main.tf lab_12 U outputs.tf U $ entry-script.sh U main.tf ../webserver U
lab_12 > $ entry-script.sh
1  #!/bin/bash
2  set -e
3  yum update -y
4  yum install -y nginx
5  systemctl start nginx
6  systemctl enable nginx
7
8  # Create directories for SSL certificates if they don't exist
9  mkdir -p /etc/ssl/private
10 mkdir -p /etc/ssl/certs
```

Apply Terraform: Terraform apply -auto-approve

```
terraform apply -auto-approve
webserver_public_ip = "3.29.63.209"
```

View output: Terraform output

```
webserver_public_ip = "3.29.63.209"
```

Verify HTTPS page



Verify HTTP to HTTPS redirect



Task 7: Configure Nginx as reverse proxy

Create Apache backend script

```
main.tf lab_12 U outputs.tf U $ entry-script.sh U $ apache.sh U × main.tf ..
lab_12 > $ apache.sh
1  #!/bin/bash
2  yum update -y
3  yum install httpd -y
4  systemctl start httpd
5  systemctl enable httpd
6  echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html
7  hostnamectl set-hostname myapp-webserver
8  echo "<h2>Hostname: $(hostname)</h2>" >> /var/www/html/index.html
9  TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
10 -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

Add backend webserver module

```
main.tf lab_12 U × outputs.tf U $ entry-script.sh U $ apache.sh U main.tf ..
lab_12 > main.tf
35 module "myapp-web-1" {
36   source = "../modules/webserver"
37   env_prefix = var.env_prefix
38   instance_type = var.instance_type
39   availability_zone = var.availability_zone
```

Update outputs

```
main.tf lab_12 U outputs.tf U × $ entry-script.sh U $ apache.sh U main.tf ..
lab_12 > outputs.tf > ...
1  output "webserver_public_ip" {
2    value = module.myapp-webserver.aws_instance.public_ip
3  }
4
5  output "aws_web-1_public_ip" {
6    value = module.myapp-web-1.aws_instance.public_ip
7  }
```

Apply Terraform: Terraform apply -auto-approve

```
module.myapp-web-1.aws_instance.myapp-server: Creation complete after 12s [id=i-0e678ed6bf61983e3]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:

aws_web-1_public_ip = "51.112.180.9"
webserver_public_ip = "3.29.63.209"
```

5. View outputs: Terraform output

```
aws_web-1_public_ip = "51.112.180.9"
webserver_public_ip = "3.29.63.209"
```

6. SSH into nginx server: Ssh [ec2-user@3.29.63.209](ssh://ec2-user@3.29.63.209)


```
[ec2-user@ip-10-0-10-65 ~]$ cat /etc/ssl/certs/selfsigned.crt
```

10. Verify reverse proxy in browser



```
lab_12 > main.tf > ...
48 module "myapp-web-2" {
49   source = "../modules/webserver"
50   env_prefix = var.env_prefix
51   instance_type = var.instance_type
52   availability_zone = var.availability_zone
53   public_key = var.public_key
54   my_ip = local.my_ip
55   vpc_id = aws_vpc.myapp_vpc.id
```

```
lab_12 > outputs.tf > ...
9   output "aws_web-2_public_ip" {
10     value = module.myapp-web-2.aws_instance.public_ip
11   }
```

Apply Terraform: Terraform apply -auto-approve

```
aws_web-1_public_ip = "51.112.180.9"  
aws_web-2_public_ip = "3.28.204.85"  
webserver_public_ip = "3.29.63.209"
```

```
A newer release of "Amazon Linux" is available.
Version 2023.10.20260105:
Version 2023.10.20260120:
Version 2023.10.20260120:
Run "/usr/bin/dnf check-release-update" for full release and version update info

  #
~\_ ##### Amazon Linux 2023
~\_ \#####\
~\_ \###|
~\_ \#/ https://aws.amazon.com/linux/amazon-linux-2023
~\_ V~' '~>
~\_
~\_ _ _ _ _
~\_ / / / /
~\_ /m/'

Last login: Sat Jan 24 10:47:07 2026 from 4.240.39.196
[ec2-user@ip-10-0-10-65 ~]$
```

```
upstream backend_servers {
    server 51.112.180.9:80;
    server 3.28.204.85:80;
}

server {
    listen 443 ssl;
    server_name 3.29.63.209;
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        proxy_pass http://51.112.180.9:80;
        proxy_pass http://backend_servers;
    }
}
```

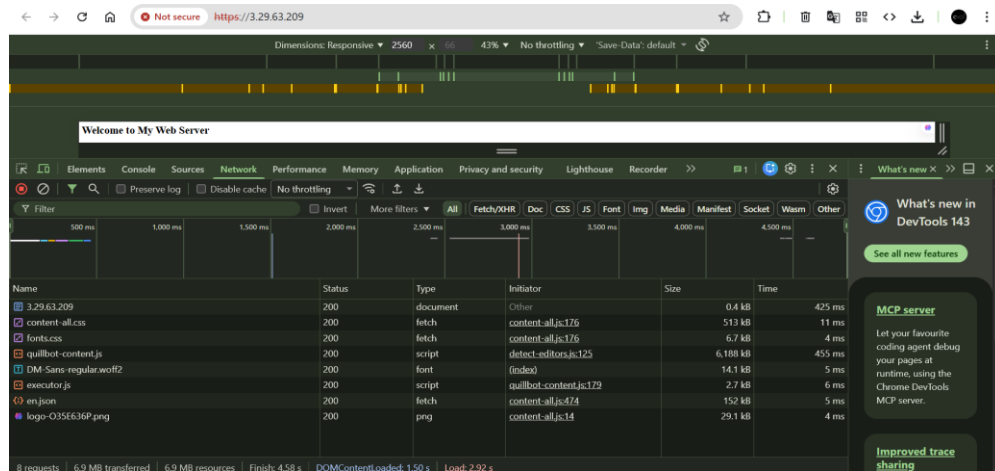
```
[ec2-user@ip-10-0-10-65 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-65 ~]$
```

1. Configure web-1 primary
Ssh ec2-user@3.29.63.209

Restart nginx

```
[ec2-user@ip-10-0-10-65 ~]$ sudo systemctl reload nginx
[ec2-user@ip-10-0-10-65 ~]$
```

Verify cache MISS



Cleanup

1. Destroy all resources:Terraform destroy

```
module.myapp-web-2.aws_instance.myapp-server: Destruction complete after 1m10s
module.myapp-web-2.aws_key_pair.ssh-key: Destroying... [id=dev-serverkey-2]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destroying... [id=subnet-0b3d482b94bd8991f]
module.myapp-web-2.aws_security_group.web_sg: Destroying... [id=sg-02ba95d5ac6d17798]
module.myapp-web-2.aws_key_pair.ssh-key: Destruction complete after 1s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 1s
module.myapp-web-2.aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0ebdc0b6cd81f4faa]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 13 destroyed.
```

Verify state: Cat terraform.tfstate

```
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 93,
  "lineage": "4458b676-6ac1-1909-111c-b68a95405807",
  "outputs": {},
  "resources": [],
  "check_results": null
}
```

Verify final files: tree

